

# Генератор тестовых РЧ сигналов (КВ)

Веб-интерфейс для трансляции библиотеки тестовых IQ-сигналов в КВ-диапазон через **HackRF**

**One.** Выбираешь сигнал из выпадающего списка → задаёшь частоту/усиление → жмёшь «Старт» → сигнал идёт в эфир, а в браузере в реальном времени рисуется его спектр и водопад. Есть предпросмотр спектра без передачи в эфир, и обмен сигналами между экземплярами программы через файлы-бандлы.

Оформление — чёрно-зелёная терминальная тема.

---

## Содержание

---

1. Структура проекта
  2. Общая архитектура и поток данных
  3. Как это устроено — по шагам
    - Формат хранения сигналов
    - Конвертация WAV → IQ
    - Генерация тестового тона
    - Кэш спектра
    - Передача в эфир
    - Живой спектр и предпросмотр
    - Экспорт и импорт бандлов
  4. Схема library.json
  5. API бэкенда
  6. Установка и запуск
  7. Добавление сигналов в библиотеку
  8. Обмен сигналами между машинами
  9. Веб-интерфейс
  10. Перенос на Raspberry Pi
  11. История отладки и известные грабли
  12. Безопасность по уровню TX
-

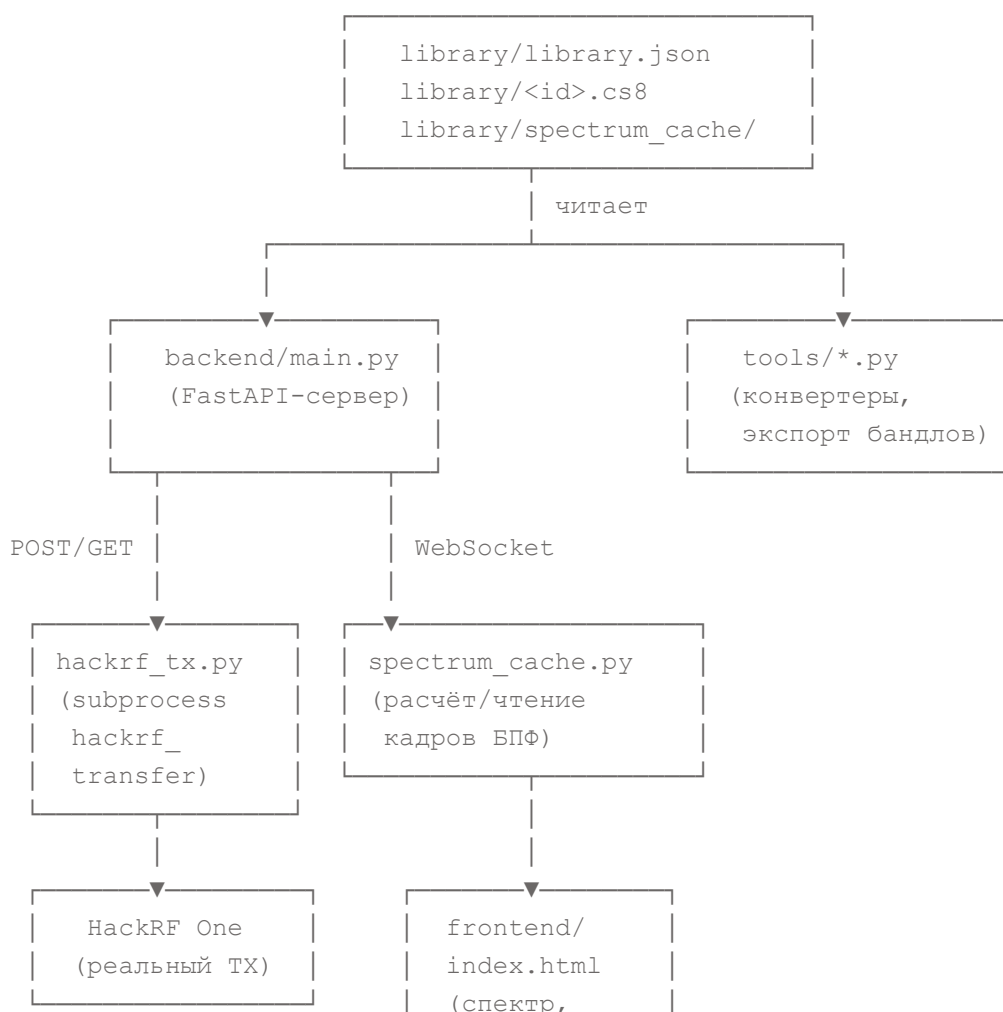
## Структура проекта

```

iq_broadcast/
  backend/
    main.py          # FastAPI: все эндпоинты, состояние трансляции, потоки
    hackrf_tx.py     # обёртка над hackrf_transfer (запуск/остановка процесса
    spectrum_cache.py # расчёт БПФ + кэширование, используется и live, и офлайн
    library_utils.py # чтение/запись library.json (общее для конвертеров и ма
  tools/
    wav_to_iq_library.py # конвертер: WAV -> .cs8 + запись в библиотеку + кэш спек
    generate_sine.py     # генератор чистого тона -> .cs8 + запись в библиотеку
    export_bundle.py     # упаковка сигналов библиотеки в .tar для передачи друг
  library/
    library.json        # метаданные всех сигналов
    <id>.cs8             # сами IQ-файлы (int8, готовые для HackRF)
    spectrum_cache/
      <id>.npz           # предсчитанный спектр (кадры БПФ), см. ниже
  frontend/
    index.html          # весь веб-интерфейс: разметка + стили + JS одним файлом
  requirements.txt
  README.md

```

## Общая архитектура и поток данных



|                         |
|-------------------------|
| водопад,<br>управление) |
|-------------------------|

Два принципиально независимых потока данных:

- **Тракт передачи:** `main.py` запускает `hackrf_transfer` (через `hackrf_tx.py`), который читает `.cs8`-файл с диска и льёт его в HackRF. Никакого Python на пути самих сэмплов нет — это оказалось критично для стабильности (см. [историю отладки](#)).
- **Тракт визуализации:** отдельный поток читает тот же `.cs8` (либо готовый кэш) и считает/шлёт спектр по WebSocket. Он никак не связан с трактом передачи — падение одного не затрагивает другой, и можно даже смотреть предпросмотр совсем другого сигнала во время реальной трансляции.

---

## Как это устроено — по шагам

---

### 1. Формат хранения сигналов

Каждый сигнал в библиотеке — это готовый **int8 IQ-файл** (`.cs8`, интерливинг I,Q по одному байту на компоненту, диапазон -128..127) уже на той частоте дискретизации, на которой его будет читать `hackrf_transfer` (по умолчанию 2 МГц). Никакого промежуточного "мастер-формата" и рендера при воспроизведении не делается — конвертация в формат HackRF происходит один раз, при добавлении сигнала в библиотеку, инструментами из `tools/`.

Это было не всегда так — изначально в библиотеке хранился компактный "мастер"-файл на низкой частоте (200 кГц, int16), а передискретизация под HackRF происходила на лету при каждом запуске воспроизведения. От этой схемы отказались: рендер занимал заметное время (около минуты на 3-минутный сигнал) прямо перед стартом передачи. Раз конвертация всё равно нужна один раз — логичнее сразу писать целевой формат и не тратить время при каждом `play`. Расплата — файлы в библиотеке крупнее (могут быть сотни МБ на длинную запись), но это одноразовая плата при добавлении сигнала, а не при каждом воспроизведении.

### 2. Конвертация WAV → IQ (`wav_to_iq_library.py`)

Берёт моно WAV-запись HF-сигнала таким, каким он звучит в наушниках после демодуляции в SSB (тона модема на аудио-частотах, обычно 0.3–3 кГц — подходит для STANAG-4285, ALE, PACTOR, RTTY, PSK31 и т.п.), и превращает её в комплексный IQ-сигнал, готовый для передачи через SDR. Шаги конвейера:

1. **Чтение WAV**, приведение к моно (берётся первый канал, если их несколько) и нормализация амплитуды в диапазон [-1, 1].
2. **ФНЧ (Баттерворт, 6 порядка)** перед преобразованием Гильберта — срез по умолчанию 3500 Гц. Убирает шум у границы Найквиста исходной записи: без этого шага при построении аналитического сигнала (следующий шаг) такой шум просачивается зеркальным артефактом на отрицательные частоты из-за эффекта Гиббса на конечном по длине преобразовании.

3. **Преобразование Гильберта** — из вещественного аудио-сигнала строит аналитический (комплексный) сигнал. Это ключевой шаг: вещественная запись сама по себе не несёт информации о том, по какую сторону от несущей должен оказаться сигнал при передаче; преобразование Гильберта восстанавливает эту информацию, воспроизводя ту же боковую полосу (USB/LSB), в которой сигнал изначально принимался.
4. **Передискретизация** с исходной частоты WAV (обычно 8–16 кГц) на целевую частоту библиотеки (по умолчанию 2 МГц) — линейная интерполяция блоками, без построения полноценного полифазного FIR-фильтра (при таком огромном коэффициенте передискретизации, в сотни раз, полноценный FIR создал бы неприемлемый всплеск потребления памяти). Передискретизация идёт сразу в целевой формат (int8), без промежуточного файла.
5. **Масштабирование и запись** — амплитуда домножается на коэффициент `--gain` (0..1, по умолчанию 0.7, чтобы был запас от клиппинга при переводе в 8 бит) и пишется как interleaved int8 I/Q.
6. **Регистрация в библиотеке** — запись метаданных в `library.json` (через `library_utils.register_signal`, заменяет запись с тем же `id`, если она уже была).
7. **Построение кэша спектра** — автоматически вызывается `spectrum_cache.build_cache_for_signal()` (см. ниже), отдельного шага не требуется.

Полный список параметров командной строки:

| Флаг                       | По умолчанию            | Смысл                                             |
|----------------------------|-------------------------|---------------------------------------------------|
| <code>input_wav</code>     | —                       | путь к исходному WAV (обязательный)               |
| <code>--id</code>          | —                       | уникальный ID в библиотеке (обязательный)         |
| <code>--name</code>        | —                       | отображаемое имя (обязательный)                   |
| <code>--freq</code>        | —                       | рекомендуемая частота передачи, Гц (обязательный) |
| <code>--description</code> | <code>""</code>         | описание сигнала                                  |
| <code>--library</code>     | <code>../library</code> | путь к папке библиотеки                           |
| <code>--sample-rate</code> | <code>2000000</code>    | частота дискретизации итогового файла, Гц         |
| <code>--gain</code>        | <code>0.7</code>        | масштаб амплитуды 0..1                            |
| <code>--lowpass</code>     | <code>3500.0</code>     |                                                   |

| Флаг                                 | По умолчанию        | Смысл                                                                                           |
|--------------------------------------|---------------------|-------------------------------------------------------------------------------------------------|
|                                      |                     | срез ФНЧ перед Гильбертом, Гц<br>( <code>0</code> — отключить)                                  |
| <code>--tx-vga-gain</code>           | <code>20</code>     | TX VGA gain HackRF по умолчанию для этого сигнала, 0–47 дБ                                      |
| <code>--amp-enable</code>            | выкл                | включить встроенный усилитель HackRF (+14 дБ) по умолчанию                                      |
| <code>--sideband</code>              | <code>usb</code>    | боковая полоса исходной записи<br>( <code>usb</code> / <code>lsb</code> ) — информационное поле |
| <code>--spectrum-freq-min-khz</code> | <code>-5.0</code>   | нижняя граница окна спектра в интерфейсе, кГц                                                   |
| <code>--spectrum-freq-max-khz</code> | <code>5.0</code>    | верхняя граница окна спектра, кГц                                                               |
| <code>--spectrum-db-min</code>       | <code>-100.0</code> | нижняя граница шкалы амплитуды на графике, дБ                                                   |
| <code>--spectrum-db-max</code>       | <code>0.0</code>    | верхняя граница шкалы амплитуды, дБ                                                             |
| <code>--loop</code>                  | вкл                 | зациклить воспроизведение по умолчанию для этого сигнала                                        |

Пример:

```
cd tools
python3 wav_to_iq_library.py path/to/signal.wav \
  --id my-signal \
  --name "Название сигнала" \
  --freq 7000000 \
  --description "..." \
  --library ../library
```

### 3. Генерация тестового тона ( `generate_sine.py` )

Отдельный, гораздо более простой путь для чистых калибровочных сигналов — без WAV на входе, без ФНЧ и Гильберта. Генерирует комплексную экспоненту напрямую:

- При `--offset-khz 0` — чистая несущая: постоянный вектор  $I = \text{amplitude}, Q = 0$  на всю длительность. Шва при зацикливании быть не может в принципе (сигнал константен).
- При ненулевом смещении —  $I(t) + jQ(t) = \text{amplitude} \cdot e^{(j \cdot 2\pi \cdot \text{offset} \cdot t)}$ . Длительность автоматически подгоняется под **целое число периодов** тона, чтобы при

заикливания файла ( `hackrf_transfer -R` ) не было щелчка на стыке (последний отсчёт файла плавно продолжается первым).

Смещение от центра по умолчанию **+1 кГц, не 0** — у передатчиков прямого преобразования (в том числе HackRF) на самой несущей нередко виден паразитный DC-спайк от утечки гетеродина; тон в стороне от центра с ним не спутать на анализаторе спектра.

Параметры командной строки:

| Флаг                                                                           | По умолчанию                                             | Смысл                                                                                                 |
|--------------------------------------------------------------------------------|----------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| <code>--id</code>                                                              | —                                                        | уникальный ID<br>(обязательный)                                                                       |
| <code>--name</code>                                                            | "Тестовый тон"                                           | отображаемое имя                                                                                      |
| <code>--freq</code>                                                            | —                                                        | центральная частота<br>передачи, Гц<br>(обязательный)                                                 |
| <code>--offset-khz</code>                                                      | 1.0                                                      | смещение тона от<br>центра, кГц ( 0 =<br>чистая несущая)                                              |
| <code>--amplitude</code>                                                       | 0.8                                                      | амплитуда 0..1                                                                                        |
| <code>--duration</code>                                                        | 2.0                                                      | ориентировочная<br>длительность, с<br>(реальная будет<br>скорректирована под<br>целое число периодов) |
| <code>--sample-rate</code>                                                     | 2000000                                                  | частота дискретизации,<br>Гц                                                                          |
| <code>--library</code>                                                         | ../library                                               | путь к библиотеке                                                                                     |
| <code>--tx-vga-gain</code>                                                     | 20                                                       | TX VGA gain по<br>умолчанию                                                                           |
| <code>--amp-enable</code>                                                      | выкл                                                     | усилитель HackRF по<br>умолчанию                                                                      |
| <code>--description</code>                                                     | автогенерируется                                         | описание                                                                                              |
| <code>--spectrum-freq-min-khz</code> /<br><code>--spectrum-freq-max-khz</code> | подбирается<br>автоматически (с<br>отступом вокруг тона) | окно спектра                                                                                          |
| <code>--spectrum-db-min</code> /<br><code>--spectrum-db-max</code>             | -100.0 / 0.0                                             | шкала амплитуды                                                                                       |

Пример:

```
python3 generate_sine.py --id carrier-10mhz --name "Несущая 10 МГц" \
    --freq 10000000 --offset-khz 0 --library ../library
```

#### 4. Кэш спектра ( `spectrum_cache.py` )

Расчёт БПФ по всему файлу сигнала — не самая дешёвая операция, а при зацикленном воспроизведении она бы повторялась заново на каждый круг без всякой необходимости (сигнал в файле один и тот же). Поэтому спектр считается **один раз, при добавлении сигнала в библиотеку**, и кэшируется:

- `iter_raw_frames(file)` — читает файл окнами по `CHUNK_SAMPLES = 65536` отсчётов, на каждом окне считает БПФ с окном Блэкмана-Харриса (низкие боковые лепестки) и отдаёт спектр в дБ. Последний неполный блок в конце файла отбрасывается. Этот генератор — **общий код**, который используют и построение кэша, и живой расчёт (если кэша нет) — поэтому поведение между ними гарантированно не расходится.
- `build_cache_for_signal(signal_meta, library_dir)` — прогоняет `iter_raw_frames` по всему файлу, обрезает каждый кадр по частоте (`spectrum_freq_min_khz / max_khz` из метаданных сигнала) и сохраняет всё в `library/spectrum_cache/<id>.npz` (сжатый numpy-архив: массив частот, матрица кадров `[число_кадров × число_точек]`, плюс метаданные для проверки валидности).
- `load_cache_if_valid(signal_meta, library_dir)` — при воспроизведении проверяет, что кэш существует, не старше самого `.cs8` (по mtime) и совпадает по `sample_rate`, размеру окна БПФ и границам обрезки с текущими метаданными сигнала. При любом несовпадении — тихо возвращает `None`, без исключений, и код-потребитель просто считает вживую.

Кэш **не кладётся ни в бандлы при экспорте, ни куда-либо ещё** — он регенерируется на месте (в том числе автоматически при импорте бандла), чтобы не завязываться на версию `spectrum_cache.py` на другой машине.

Усреднение (сглаживание спектра между кадрами, экспоненциальное, `AVG_ALPHA = 0.25`) в кэше **не хранится** — кэшируются сырые кадры, усреднение делается на лету при проигрывании (в `main.py`). Это сохраняет одинаковое визуальное поведение независимо от того, кэш используется или живой расчёт, включая плавный переход на стыке зацикливания.

Можно пересчитать кэш вручную, без перекодирования самого сигнала (например, если поменяли `CHUNK_SAMPLES` в коде):

```
cd backend
python3 spectrum_cache.py                # пересчитать для ВСЕХ сигналов библиоте
python3 spectrum_cache.py stanag-4285    # только для одного
```

Размер кэша по сравнению с самим IQ-файлом пренебрежимо мал: для STANAG-4285 (172 сек, 2 МГц) — кэш **4.15 МБ** против **688 МБ** самого `.cs8`.

## 5. Передача в эфир ( `hackrf_tx.py` )

`HackRFTransmitter` — тонкая обёртка вокруг штатной утилиты `hackrf_transfer` (не питоновские биндинги `libhackrf` — сама утилита от разработчиков HackRF надёжнее держит реальное время передачи).

**Запуск** ( `start()` ):

```
hackrf_transfer -t <файл> -f <частота> -s <sample_rate> -x <tx_vga_gain> -a <0|1>
```

- `-t <файл>` — читает готовый `.cs8` прямо с диска (не поток из Python — см. [историю отладки](#), почему).
- `-R` добавляется только если `loop=True` для этого запуска сигнала. Без него `hackrf_transfer` доигрывает файл один раз и завершается сам, кодом `0` — это распознаётся как штатное завершение, а не ошибка.
- Перед стартом — `_preflight_check()`: пробует `hackrf_info` из того же комплекта, что и `hackrf_transfer` (ищется рядом по пути), с несколькими попытками — сразу после остановки предыдущей передачи устройство иногда не успевает освободиться мгновенно.
- На Windows процесс запускается в **своей отдельной, скрытой консоли** ( `CREATE_NEW_CONSOLE` + `STARTUPINFO` с `SW_HIDE` ) — технически нужна для корректной остановки (см. ниже), но визуальное окно не видно.

**Остановка** ( `close()` → `_graceful_stop()` ): жёсткий `terminate()` не подходит — на Windows это `TerminateProcess()`, который не даёт `hackrf_transfer` шанса корректно освободить устройство ( `hackrf_stop_tx()` → `hackrf_close()` → `hackrf_exit()` ), из-за чего HackRF остаётся в состоянии "передача идёт" на уровне прошивки, и следующий `hackrf_open()` может провалиться, пока не переподключишь USB физически. Вместо этого:

- на Linux/macOS — обычный `SIGINT` (то же самое, что нажать Ctrl+C);
- на Windows — отправка `CTRL_C_EVENT` через **отдельный короткоживущий вспомогательный процесс** ( `python -c "..."` ), который подключается к консоли `hackrf_transfer` ( `AttachConsole` ), шлёт событие и сразу завершается. Такая возня именно в отдельном процессе, а не в самом сервере — принципиальна: `AttachConsole` / `FreeConsole` работают на уровне всего процесса, а не потока, и если делать это прямо в потоке внутри `uvicorn`, на короткое время **весь** сервер (включая обработку HTTP-запросов в других потоках) остаётся без консоли и может подвиснуть.
- если корректная остановка не сработала за отведённое время — тогда уже `terminate()` / `kill()` как крайняя мера.

**Мониторинг** ( `check_alive()` ): при неожиданном завершении процесса с ненулевым кодом — бросает исключение (реальный сбой, например обрыв USB); с кодом `0` — считает штатным завершением (актуально при `loop=False`).

Если бинарник `hackrf_transfer` не найден в `PATH` (и не задан явно через переменную окружения `HACKRF_TRANSFER_BIN`) — класс переходит в режим **симуляции**: не запускает никакого процесса, а просто выдерживает паузы в реальном времени, как будто сэмплы уходят в эфир. Удобно для разработки интерфейса на машине без подключённого HackRF.

## 6. Живой спектр и предпросмотр

Обе задачи используют одну и ту же функцию

`stream_spectrum(signal_meta, emit_fn, stop_event)` в `main.py` — она либо проигрывает кадры из кэша по кругу, либо (если кэша нет) считает вживую через `iter_raw_frames`, с тем же усреднением и тем же реальным темпом (пауза между кадрами рассчитывается исходя из фактического времени обработки, чтобы не накапливать дрейф). Единственное, что отличается между двумя режимами использования — куда деваются посчитанные кадры и что их останавливает:

- **Реальная трансляция** (`spectrum_worker`) — кадры уходят `state.broadcast()`, то есть всем клиентам, подключённым к `/ws/spectrum`; останавливается общим `state.stop_flag` (тем же, что останавливает и саму передачу через HackRF).
- **Предпросмотр** (`/ws/preview/{signal_id}`) — на каждое WebSocket-подключение поднимается свой поток и свой `stop_event`, кадры уходят напрямую в это одно соединение. Полностью независим от `PlaybackState` — не трогает HackRF и не влияет на реальную трансляцию (если она в этот момент идёт). Можно одновременно транслировать один сигнал и просматривать спектр совершенно другого — они друг другу не мешают. Поток завершается, когда клиент закрывает WebSocket (закрывает модальное окно предпросмотра в интерфейсе).

Перед отправкой спектр обрезается по частоте согласно

`spectrum_freq_min_khz`/`max_khz` сигнала — на клиент уходит уже готовый, компактный набор точек (обычно 150–300 в зависимости от ширины окна), подписи осей на графике берутся из фактических границ этих данных.

## 7. Экспорт и импорт бандлов

Формат для передачи сигналов между разными установками программы — `.tar`-архив:

```
bundle.tar
├─ manifest.json      — список метаданных сигналов (тот же формат, что в libra
├─ signals/
│   ├── <id-1>.cs8
│   └── <id-2>.cs8
```

Кэш спектра в архив **не кладётся** (см. выше — регенерируется на месте у получателя).

**Экспорт** (`tools/export_bundle.py`) — читает `library.json`, для каждого запрошенного (или всех) сигнала копирует его `.cs8` в архив под `signals/<file>` и собирает список метаданных в `manifest.json` внутри того же архива.

**Импорт** (`POST /import` в `main.py`, вызывается из интерфейса — кнопка «+ Импорт»):

1. Загруженный `.tar` сохраняется во временную директорию.
2. Распаковывается с проверкой безопасности ( `_safe_extract` ): для каждого элемента архива проверяется, что итоговый путь не выходит за пределы директории распаковки (защита от **path traversal** — `../../../etc/...` ), и что среди элементов нет символических ссылок. На Python 3.12+ дополнительно используется штатный фильтр `tarfile.extractall(filter="data")` (PEP 706) как второй уровень защиты.
3. Читается `manifest.json` , для каждой записи проверяются обязательные поля ( `id` , `name` , `file` , `sample_rate` , `recommended_freq_hz` ) — при нехватке хоть одного возвращается ошибка с указанием, каких именно полей не хватает. Отсутствующие необязательные поля (`gain`, `sideband`, `spectrum_*` и т.д.) заполняются значениями по умолчанию.
4. `.cs8` -файл сигнала копируется в `library/` под именем `<id>.cs8` (имя всегда берётся от `id` , а не от произвольного имени файла из манифеста — дополнительная защита от подмены пути).
5. Запись добавляется в `library.json` ( `register_signal` — заменяет существующую запись с тем же `id` , если она уже была; в ответе помечается `updated: true` / `false` соответственно).
6. Кэш спектра **строится заново** на месте ( `build_cache_for_signal` ).

Ответ фронтенду — список импортированных сигналов с пометкой, новый он или обновлённый; список сигналов в интерфейсе обновляется автоматически.

## Схема library.json

Массив объектов, каждый описывает один сигнал:

| Поле                             | Тип    | Обязательное | Смысл                                                                                  |
|----------------------------------|--------|--------------|----------------------------------------------------------------------------------------|
| <code>id</code>                  | строка | да           | уникальный идентификатор, используется и как имя файла ( <code>&lt;id&gt;.cs8</code> ) |
| <code>name</code>                | строка | да           | отображаемое имя в интерфейсе                                                          |
| <code>file</code>                | строка | да           | имя <code>.cs8</code> -файла в папке <code>library/</code>                             |
| <code>sample_rate</code>         | число  | да           | частота дискретизации файла, Гц                                                        |
| <code>recommended_freq_hz</code> | число  | да           | рекомендуемая частота передачи, Гц                                                     |

| Поле                               | Тип           | Обязательное | Смысл                                                                                                         |
|------------------------------------|---------------|--------------|---------------------------------------------------------------------------------------------------------------|
|                                    |               |              | (переопределяется в интерфейсе перед стартом)                                                                 |
| <code>tx_vga_gain</code>           | целое<br>0–47 | нет (20)     | TX VGA gain HackRF по умолчанию, дБ                                                                           |
| <code>amp_enable</code>            | bool          | нет (false)  | включать ли встроенный усилитель HackRF (+14 дБ) по умолчанию                                                 |
| <code>sideband</code>              | строка        | нет          | <code>usb</code> / <code>lsb</code> / <code>n/a</code> — информационное поле, на передачу не влияет           |
| <code>gain</code>                  | число<br>0..1 | нет          | амплитуда, заложенная в сами сэмплы при конвертации — <b>не меняется на лету</b> , только пересозданием файла |
| <code>loop</code>                  | bool          | нет (true)   | зацикливать ли воспроизведение по умолчанию                                                                   |
| <code>description</code>           | строка        | нет          | описание, показывается в карточке деталей                                                                     |
| <code>duration_sec</code>          | число         | нет          | длительность файла, секунд (для отображения)                                                                  |
| <code>spectrum_freq_min_khz</code> | число         | нет (-5.0)   | нижняя граница окна спектра на графике, кГц                                                                   |
| <code>spectrum_freq_max_khz</code> | число         | нет (5.0)    | верхняя граница окна спектра, кГц                                                                             |
| <code>spectrum_db_min</code>       | число         | нет (-100.0) | нижняя граница шкалы амплитуды, дБ                                                                            |
| <code>spectrum_db_max</code>       | число         | нет (0.0)    | верхняя граница шкалы амплитуды, дБ                                                                           |

Поля `tx_vga_gain`, `amp_enable`, `loop`, `recommended_freq_hz` — это значения **по умолчанию**; интерфейс всегда даёт их переопределить перед запуском конкретной передачи, не трогая сам `library.json`.

## API бэкенда

| Метод | Путь                    | Назначение                                                                                                                                                                                                                       |
|-------|-------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| GET   | /signals                | список всех сигналов из <code>library.json</code>                                                                                                                                                                                |
| GET   | /status                 | текущее состояние: что играет, фаза ( <code>starting</code> / <code>playing</code> / <code>null</code> ), эффективные частота/gain/amp/loop, последняя ошибка                                                                    |
| POST  | /play/{signal_id}       | запустить передачу. Query-параметры (все опциональные, переопределяют значения из <code>library.json</code> только на эту сессию): <code>freq_hz</code> , <code>tx_vga_gain</code> , <code>amp_enable</code> , <code>loop</code> |
| POST  | /stop                   | остановить текущую передачу                                                                                                                                                                                                      |
| WS    | /ws/spectrum            | поток спектра активной трансляции (кадры вида <code>{"freqs": [...], "db": [...]}</code> )                                                                                                                                       |
| WS    | /ws/preview/{signal_id} | предпросмотр спектра сигнала без передачи в HackRF                                                                                                                                                                               |
| POST  | /import                 | импорт бандла ( <code>multipart/form-data</code> , поле <code>file</code> — <code>.tar</code> )                                                                                                                                  |

Фаза (`phase`) в `/status` — `"starting"` появляется сразу после запуска потока передачи (пока идёт preflight-проверка устройства и старт `hackrf_transfer`, это может занять пару секунд), затем сменяется на `"playing"`. Фронтенд ждёт именно `null`, чтобы считать передачу остановленной — на промежуточных фазах кнопка «Стоп» остаётся активной.

## Установка и запуск

```
pip install -r requirements.txt
```

`hackrf_transfer` (входит в комплект `hackrf` / PothosSDR) должен быть доступен в `PATH`, либо укажите путь явно переменной окружения:

- Windows (PothosSDR):

```
set HACKRF_TRANSFER_BIN=D:\Program Files\PothosSDR\bin\hackrf_transfer.exe
```

- Linux / Raspberry Pi: `sudo apt install hackrf` — обычно сразу попадает в `PATH`.

Если `hackrf_transfer` не найден — бэкэнд автоматически переходит в режим симуляции (см. выше), интерфейс при этом работает полностью, просто без реальной передачи.

Запуск:

```
cd backend
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

Открыть `http://localhost:8000`.

## Частота дискретизации HackRF

Разные сборки `hackrf_transfer` поддерживают разный диапазон: старые (в т.ч. которая идёт в PothosSDR) — иногда только фиксированный набор 8/10/12.5/16/20 МГц; более новые — произвольные значения в диапазоне 2–20 МГц. **2 МГц проверено вручную и работает стабильно** — это и есть значение по умолчанию у конвертеров. Более широкая полоса (для сигналов шире нескольких кГц) потребует более высокой частоты — но размер файла и время конвертации растут пропорционально.

---

## Добавление сигналов в библиотеку

Два инструмента, оба сразу и создают файл, и регистрируют сигнал, и строят кэш спектра — ничего дополнительно делать не нужно:

- Из WAV-записи HF-сигнала — `tools/wav_to_iq_library.py` (см. [подробности конвейера](#)).
- Чистый тон/несущая для калибровки — `tools/generate_sine.py` (см. [подробности](#)).

После запуска любого из них сигнал сразу появляется в выпадающем списке интерфейса — перезапускать сервер не нужно (список сигналов читается из `library.json` заново при каждом обращении к `/signals`).

---

## Обмен сигналами между машинами

Экспорт (на машине-источнике):

```
cd tools
python3 export_bundle.py --ids stanag-4285 test-tone-1khz --output bundle.tar --l
# или сразу всю библиотеку:
python3 export_bundle.py --all --output full_export.tar --library ../library
```

Импорт (на машине-получателе) — прямо через веб-интерфейс: кнопка «+ Импорт» в шапке, выбрать `.tar`-файл. Сигналы появятся в списке сразу после успешного импорта, кэш спектра посчитается автоматически. Если в библиотеке получателя уже был сигнал с таким же `id` — он будет заменён (в статусе импорта это явно помечается как «обновлён»).

## Веб-интерфейс

---

Левая панель:

- **Выпадающий список сигналов** + кнопка **«Предпросмотр»** рядом с ним — открывает модальное окно с графиком спектра выбранного сигнала, проигрываемым в цикле, без передачи в эфир. Закрывается крестиком, кликом по фону или клавишей Escape.
- **Карточка деталей** — рекомендуемая частота, длительность, частота дискретизации, боковая полоса, амплитуда файла (с пояснением, что она зашита в файл и не меняется на лету), описание.
- **Частота передачи** — редактируемое поле, предзаполняется рекомендуемой частотой сигнала.
- **TX VGA gain** — редактируемое поле, 0–47 дБ.
- **Усилитель** и **Зациклить** — тумблеры.
- **Старт/Стоп** — внизу панели. Во время передачи весь блок настроек (список, частота, gain, тумблеры) визуальнo блокируется (тускнеет), кнопка «Предпросмотр» при этом остаётся активной — она никак не мешает идущей передаче.

Правая панель — спектр (линия, с осями и подписями) и водопад (прокручивающаяся спектрограмма, та же частотная шкала, тот же отступ слева — совпадают визуальнo). Обе — в монохромной зелёной палитре с лёгким свечением, под общий терминальный стиль интерфейса.

Шапка — заголовок и кнопка **«+ Импорт»** для загрузки бандлов сигналов.

---

## Перенос на Raspberry Pi 4 (8 ГБ) с тачскрином

---

1. Скопировать проект на Pi, `pip install -r requirements.txt`,  
`sudo apt install hackrf`.
2. Интерфейс уже адаптирован под тач: крупные кнопки, без эффектов наведения (`:hover`), обработка `touchstart`.
3. Автозапуск бэкенда — systemd-юнит:

```
# /etc/systemd/system/iq-broadcast.service
[Unit]
Description=IQ Broadcast backend
After=network.target

[Service]
WorkingDirectory=/home/pi/iq_broadcast/backend
ExecStart=/usr/bin/python3 -m uvicorn main:app --host 0.0.0.0 --port 8000
Restart=on-failure
User=pi
```

```
[Install]
WantedBy=multi-user.target

sudo systemctl enable --now iq-broadcast
```

4. Автозапуск браузера в киоск-режиме (зависит от образа Raspberry Pi OS — автологин + автостарт для LXDE/Wayfire):

```
chromium-browser --kiosk --noerrdialogs --disable-infobars http://localhost:8
```

5. Права на USB для HackRF без root: udev-правило

( `/etc/udev/rules.d/53-hackrf.rules` ), идёт в комплекте пакета `hackrf` ) и добавление пользователя `pi` в группу `plugdev` .

6. На Linux вся логика корректной остановки ( `SIGINT` ) значительно проще, чем на Windows — никакой возни с отдельными консолями не требуется, это чисто windows-специфичная часть `hackrf_tx.py` (код под неё защищён проверкой `IS_WINDOWS` и на Linux просто не выполняется).

## История отладки и известные грабли

Коротко — что было опробовано и почему отброшено, полезно держать в виду при дальнейшей доработке:

- **Потоковая подача сэмплов в `hackrf_transfer` через `stdin`** ( `-t -` ) — первый вариант передачи, обрывался через ~1 секунду на Windows с ошибкой `streaming terminated (-1004)` . Причина — недостаточно быстрая и стабильная доставка данных через Python в реальном времени. Чтение того же самого файла с диска ( `-t <file>` ) оказалось стабильным сколько угодно долго — на этом и остановились.
- **Рендер под частоту HackRF на лету при каждом `play`** (из компактного мастер-файла) — работало, но добавляло заметную задержку перед стартом каждой передачи. Заменено на прямую конвертацию в целевой формат один раз при добавлении сигнала.
- **`terminate()` для остановки `hackrf_transfer` на Windows** — жёсткое `TerminateProcess()` , не даёт процессу шанса освободить USB-устройство корректно; HackRF оставался в состоянии "передача идёт" до переподключения. Заменено на честный `CTRL_C_EVENT` через отдельный вспомогательный процесс (см. [раздел про `hackrf\_tx.py`](#)).
- **`CTRL_BREAK_EVENT` вместо `CTRL_C_EVENT`** — первая попытка честной остановки, не сработала: данная сборка `hackrf_transfer` слушает конкретно `CTRL_C_EVENT` (код `0` ), `CTRL_BREAK_EVENT` (код `1` ) её обработчиком не распознаётся.
- **`AttachConsole` / `FreeConsole` прямо в потоке backend-процесса** — вызвало полное зависание сервера: эти вызовы действуют на уровне всего процесса, а не потока, и на короткое время оставляли **весь** `uvicorn` без консоли, из-за чего логирование в других потоках подвисало. Решение — вынести всю эту возню в отдельный короткоживущий вспомогательный процесс.

- **"Гонка" при быстром stop → play** — сразу после остановки предыдущей передачи HackRF иногда не успевал освободиться, следующий запуск ловил `HackRF not found (-5)`.

Решение — `_preflight_check` с несколькими попытками и небольшой задержкой между ними.

---

## Безопасность по уровню TX

---

Начинайте с малых значений `tx_vga_gain` (например 10–15) и с выключенным `amp_enable`, поднимайте постепенно, контролируя реальную мощность на выходе антенного тракта. Значения по умолчанию в библиотеке — ориентировочные, не рассчитаны под конкретную антенну/усилитель/юридически допустимую мощность на вашей частоте.